JavaSE

运行机制：
.java 经过 javac 编译变成.class 文件（字节码文件）,通过 java 运行
运行 java.exe 对字节码文件进行执行，本质是.class 装载到 JVM 执行
细节
一个源文件只能有一个 public 的类，public 的类名必须与源文件名相同
可以将 main 写在非 public 类当中，然后制定运行非 public 类
\r 是回车符号，会来到光标当前行的最前面

JDK,JRE,JVM 的关系：
Java development kit 软件开发工具包
Java runtime environment 运行环境
Java virtual machine 一个虚构出来的计算机，引入 JVM 后，Java 语言在不同平台上运行时不需要重新编译，是跨平台的核心
JDK=JRE+开发工具；JRE=JVM+类库

配置环境变量：作用是指定命令搜索路径，在 i 命令行下面执行命令如 javac 编译 java 程序时，它会到 PATH 变量所指定的路径中查找看是否能找到相应的命令程序
在 dos 任意目录下，都可以使用 java 和 javac 命令
配置过程：系统属性——高级——环境变量

数据类型细节：
Java 数据类型在任何操作系统中都保持不变，保证了可移植性
在常量后边加‘l’‘L’表示成 long 类型
浮点数默认为 double，如果要改成 float 类型要在后面加 f
不要对运算过后的小数进行比较大小，
应该以两个数的差值的绝对值在某个精度范围内进行判断
Char 的本质是一个整数，通过 Unicode 编码转换，字符被存储时，首先转换成对应的码值，然后变成二进制存储
想把一个 char 转换成 Unicode 编码，只需要强转成 int 型
同样 char a=97，输出 a 得到的是字母 a

字符编码表：ASCII 码用一个字节表示，但是实际上只用了 128 个
Unicode 码两个字节，字母和汉字统一占两个字节
utf-8：字母用一字节，汉字用三字节

类型转换：
多种类型数据混合运算时，系统首先会转换成容量最大的数据类型
数据类型的容量排行：double>float>long>int
Byte 可以转换成 short，但是（byte,short）和 char 不能相互转化
byte,short,char 之间可进行运算，在运算时先转换成 int，比如 byte+byte 结果是 int
Boolean 不参与转换

将 int 强转成 char，对应的是 ASCII 码值
基本数据类型和 String 的转换：
基本数据转换成 String：+ “ ”
String 转换成基本数据类型：例如 Integer.parseInt(“123”)
Boolean.parseBoolean(“true”)

Java API 文档：
www.matools.com
Java 类的组织形式：JDK 下有很多包，包下有接口、类、异常、枚举
类下有字段、构造器、方法

运算符：
Int i=1;i=i++;输出 i 结果是 1
Int i=1;i=++i;输出结果是 2
关系运算符 instanceof 检查是否是类的对象，返回 true/false
短路与&&：前一个为假，后一个不判断 逻辑与&
短路或||：前一个为真，后一个不判断
逻辑异或^
复合赋值运算会进行强制类型转换：
byte b=10;b+=2;等价于 b=(byte)(b+2);因此并不会报错
b++也一样
三元运算符 int a=9;int b=10;int c=a>b?a++:b++;System.out.println(c);
由于判定式是 false，所以 a++不会被执行

```
Object obj1 = true? new Integer(1) : new Double(2.0);//三元运算符【是一个整体】 一真大师
System.out.println(obj1);// 什么? 1.0
```

三元运算符看成一个整体，Integer 被自动转换成 Double

键盘输入语句 1.导入 java.util.*2.创建该类的对象 3.调用功能
Scanner input=new Scanner(System.in);
Int a=input.nextInt();

基本语法的细节：
Switch-case 语句注意事项：
表达式数据类型，应该和 case 后的常量类型一致
返回值必须是：byte,short,int,char,enum,String(double,float,long 就不行)
case 子句中的值必须是常量，不能是变量

for 循环：
循环条件一定是一个返回 boolean 的表达式
循环初始化条件和循环结束条件可以有多条初始语句，但要求类型一样
Break 语句 通过添加标签，并写 break label，表明退出哪一层标签的循环

数组在 JVM 的内存中：数组的引用是在栈中开辟一块空间，是一个指向堆的指针

在堆中开辟一块数据类型长度*数组元素个数大小的空间，栈中的指针指向这个空间的起始位置
二维数组：是一个栈中的指针，指向堆中的某个空间，这个空间中有存放多个一维数组地址，这些一维数组地址又分别指向了一维数组（和 C 中的不一样）
声明二维数组时，各行的列数可以不等，如果列数不确定可以不写
二维数组的声明方式：int[][] y; int[] y[]; int y[][];
如果声明了一个二维数组 y[][],那么 y 表示的就是一个二维数组，而不是 y 的地址（和 C 不一样）
但是如果打印 y，输出的是 y 的地址

面向对象基础：
类和对象
类是自定义的数据类型，对象是有类创建出来的具体的东西
Jvm 中对象的存在形式：对象的实例引用存放在栈，指向堆中的空间，其中存放的是属性，如果是字符串，还要对应方法区中常量池里的区域
在一个对象 new 的过程中，要做的第一步就是在方法区加载这个类的属性信息和方法信息，然后才在栈中开辟变量名的空间
方法在 JVM 中的调用：在栈中开辟一块区域，叫做 main 栈（这个对象在 main 函数中 new 出来的），当调用到方法的时候，再在栈空间中开辟新的独立的栈
只要调用调用方法，就一定会在栈中开辟新栈
方法的调用：调用同一个类中的方法，直接写方法名；
跨类调用方法，需要通过目标类的对象来调用

重载：在同一个类中，允许多个形参列表不同的同名方法存在
重载对参数名没有要求，对返回值类型也没有要求
可变参数：将一个类中，多个同名同功能但形参列表不同的方法，封装成一个方法
使用可变参数时，可以当成数组来使用，在调用方法时，形参列表里也可以放一个数组
可变参数可以和普通参数一起放入形参列表，但要放到最后

变量的作用域(scope)：
最主要的变量就是属性（全局变量）和方法中定义的变量
属性的作用域就是整个类，局部变量只能在方法中使用
属性和局部变量可以重名，访问时就近原则；
但是同一个作用域中两个局部变量不能重名

构造器：
构造器可以重载
构造器在创建对象过程中的位置：在一个对象 new 的过程中，要做的第一步就是在方法区加载这个类的属性信息和方法信息，然后才在栈中开辟变量名的空间，空间中存放的是类的属性，先进行默认初始化，再进行显式初始化，再之后才开始进行调用构造器，最后在栈空间里用一个引用指向堆空间
所以构造器是完成对象的初始化，而不是创造对象

this 关键字：
JVM 给每个对象都分配一个 this，代表当前对象
在堆空间中，对象隐藏着一个引用，这个引用指向了对象本身的地址
this 可以访问本类的方法、属性、构造器
this 访问构造器的语法：只能在构造器中使用，用于一个构造器去访问同类中的另一个构造器，this（形参列表）；访问构造器的语法必须放在第一条语句
this 访问属性和不加 this 的区别：不加 this 是就近原则找的变量，可能是局部变量，而加 this 的是对象的属性

什么是包：
包的作用：可以区分相同名字的类，控制访问范围，方便管理
如果想要 new 一个其他包里面的类的对象，那就要在 new 的时候把包前缀写全
常用的包：java.lang.* 基本包，默认引入
java.util.* 系统提供的工具包，scanner 就属于这个包
java.net.* 网络包
java.awt.* 用于界面开发
package com.xxx;一定要在最上面，一个类中最多有一个 package

访问修饰符：用于控制方法和属性的访问范围，public 完全公开，protected 对子类和同一个包的类公开（只有不同包不能访问），缺省向同一个包的类公开（不同包的和子类都不能访问），private 只对类内公开
修饰符可以用来修饰属性、成员方法、类
只有 public 和缺省可以修饰类，而且一个.java 中只能有一个 public 的类
注意，下面图里的子类，是指的不同包里的子类，同一个包里的子类是可以访问到缺省的

1	访问级别	访问控制修饰符	同类	同包	子类	不同包
2	公开	public	✓	✓	✓	✓
3	受保护	protected	✓	✓	✓	X
4	默认	没有修饰符	✓	✓	X	X
5	私有	private	✓	X	X	X

这个图更准确

修饰符	类内部	同一个包	不同包的子类	同一个工程
private	Yes			
(缺省)	Yes	Yes		
protected	Yes	Yes	Yes	
public	Yes	Yes	Yes	Yes

面向对象三大特征：
封装
隐藏实现细节，检验数据和理性
实现步骤：将属性进行私有化，提供 public 的 set/get 方法，用于对属性判断和赋值/获取
继承
解决代码复用，从类中抽象出父类，在父类中定义方法和属性
1. 子类继承父类之后，父类私有的方法只能通过父类提供的公共的 get 方法来调用

2. 子类必须调用父类的构造器，完成父类的初始化（隐藏了 super();）
3. 调用子类时，不管使用子类哪个构造器，都默认使用父类无参构造器，如果父类没有无参构造器，就需要用在子类构造器中用 super 去指定，否则编译不通过
4. super 在使用时，放在第一行；super 和 this 都只能放在第一行，所以这两个不能同时存在
5. Java 所有类都是 object 的子类
6. 父类构造器的调用不限于直接父类
7. Java 是单继承的

继承的本质是一种查找关系
在 JVM 中，new 一个子类对象时，先在方法区加载继承关系，再在堆中分配空间（从最靠前的父类开始），所以堆中的新空间不仅有子类的属性，还会有所有父类的属性
当查找属性时，首先看子类有没有这个属性，如果有就返回子类的，没有就返回父类的

super 关键字，代表父类的引用，用于访问父类的构造器、属性、方法
不能访问父类私有的方法和属性
当子类和父类中有重名的方法或属性时，访问父类的必须通过 super
使用 super 遵循就近原则
super 和 this 的比较：
访问属性 this 优先访问本类，没有则访问父类；super 先访问父类
调用方法 和访问属性类似
调用构造器 都必须放在构造器首行

方法重写：
子类的方法和父类的方法名称、返回值、形参列表一样，就说子类的方法重写了父类的方法
名称和形参列表必须完全一样
子类的返回类型和父类一样，或者子类返回类型是父类返回方法的子类
子类方法不能缩小父类方法的访问权限
原因：假设 其他包中的对象 C 调用 方法为：get(A a){
```
        a.setXXX();
}
```
而此时传入的对象为 B 类对象，假设为 b 此时 b 将转型为 a 但是 b 中的 setXXX() 调用权限已经被缩小了这将造成错误。所以子类对象不能比父类对象访问权限大

多态
多态的表现：重载和重写体现方法的多态
对象的多态：一个对象的编译类型和运行类型可以不一样 Animal a=new Dog();编译对象是 Animal，运行对象是 Dog；编译类型在定义对象时候就确立了，不能改变，而运行类型可以改变，a=new Cat();此时运行类型就变成了 Cat
向上转型：父类的引用指向子类的对象，可调用父类的所有成员，不能调用子类的特有成员
向下转型：将父类的引用强转成子类的引用，强转之后可以调用子类所有成员
属性不存在重写，所以运行时属性由编译类型决定
Instanceof 运算符，判断一个对象是不是某个类或某个类的子类，它判断的是运行类型

动态绑定机制

当调用对象方法时，该方法会和该对象内存地址/运行类型绑定；
调用对象属性时，不存在动态绑定，哪里声明哪里使用

多态数组：数组的定义类型为父类类型，保存的实际元素为子类类型
当要调用数组元素独有的方法时，先用 instanceof 判断，再向下转型
多态参数：方法定义中的参数为父类类型，实际传递参数为子类类型

object 类：
‘==’：只要两个对象地址相同，哪怕一个是父类一个是子类，也返回 true
equals 只能用于引用类型，String 的 equals 方法被重写，变成了比较两字符串的值是否相同
Integer 也被重写
hashCode，提高哈希表的性能
toString，放回全类名+@+16 进制 hashcode
finalize：在销毁对象前，会调用 finalize 方法，程序员就可以重写 finalize 方法，加入业务逻辑代码，比如释放资源、数据库连接、或者打开文件

类变量：
也叫静态变量，在 JVM 中随着类的加载而存在于堆中，不创建类对象也可以被访问
访问时的规范方法是类名.类变量名

类方法：
在方法中不涉及和对象相关的成员，可以用类方法提高开发效率
不创建实例，就可以使用方法
类方法和普通方法都是随着类的加载而加载，存在于方法区
非静态方法可以访问非静态成员和静态成员，静态方法只能访问静态成员
main 方法的语法：
main 方法由 JVM 调用
由于调用 main 方法时不能创建对象，所以必须是 static 的
String[] args 里面的字符串数组的参数，是在执行 main 函数时传进去的
因为 main（）是静态方法，所以他可以使用所在类的静态方法和静态属性
如果要访问类中的非静态方法，需要创建对象才行

代码块
又称初始化块，属于类的一部分，是在加载类或者创建对象时隐式调用
根据位置和声明不同，可以分为局部代码块，构造代码块，静态代码块，同步代码块
局部代码块：在方法中出现，限定变量生命周期，及早释放，提高内存利用率
构造代码块（初始化块）:在类中方法外出现
静态代码块：一般用于加载驱动

分为 static 的静态代码块和非静态代码块
相当于另外形式的构造器，用于初始化，代码块调用的优先级优于构造器
静态代码块随着类的加载而执行，并且只会执行一次；普通代码块每创建一个对象就会执行一次
类被加载的情况：1.创建对象 2.创建子类对象是，父类也会被加载，而且父类先被加载 3.

使用类的静态属性/静态方法时
如果只是使用类的静态成员，普通代码块不会被执行

创建一个对象时的顺序：1.静态代码块和静态属性，如果两者都有，那么按照生命顺序
2. 普通代码块和普通属性 3.构造器
普通代码块在构造器的调用位于 super();和执行当前构造器内容之间
创建一个子类时调用顺序：1.父类的静态属性和静态代码块 2.子类的静态属性和静态代码块 3.父类的普通代码块和普通属性初始化 4.父类构造器 5.子类普通代码块和普通属性初始化 6.子类构造器

单例设计模式：
采用一定的方法，保证在软件中某个类只能有一个对象，并且该类只提供一个取得其对象实例的方法
如果某个类非常重要但非常消耗资源，只需要一个，那么单例设计模式就很有价值
饿汉式：1.构造器私有化（防止直接 new） 2.类的内部创建对象 3.向外暴露一个静态的公共方法
懒汉式：1.构造器私有化 2.定义一个 static 静态属性变量 3.提供一个公共的 static 方法，可以返回一个对象
饿汉式不存在线程安全问题

final 关键字：
final 的类不可被继承
父类中 final 的方法不可被子类重写
类中的 final 属性不可被修改
final 的局部变量不可被修改
final 修饰的变量是常量，必须赋初值，可以在定义时/代码块中/构造器中赋初值（如果 final 修饰的是 static 的，那就不能在构造器中赋值）
final static 的属性不会导致类加载

抽象类：
父类的某些方法需要声明，但不知道如何实现时，可以声明为抽象方法，这个类就成了抽象类
抽象类不能被实例化，也不一定有抽象方法
abstract 只能修饰类和方法，不能修饰其他的
继承抽象类必须实现所有抽象方法，或者自己也是抽象的
abstract 的不可以被 private、static、final 修饰
p402 抽象模板设计模式没很懂

接口 interface：
类需要实现接口声明/规定的方法
接口就是一些没有实现的方法，封装到一起，到某个类要是用的时候，再根据具体情况实现出来
jdk7 之前接口中只能有抽象方法，jdk8 之后接口可以有静态方法、默认方法，普通方法需要使用 default 关键字修饰，默认方法不用加

接口中的方法默认都是 public、abstract 的
抽象类实现接口，可以不用实现接口里的抽象方法
一个类可以同时继承多个接口
接口中的属性，默认是 public static final 的
一个接口可以继承多个其他接口
接口的修饰符只能是 public 和缺省，和类一样
接口类型的变量可以指向实现了接口的类的对象，既然接口有多态性，那也能实现接口数组，也存在动态绑定
多态传递：有个 interface A，如果类 B implements A，类 C 再 extends B，那么 C 也相当于 implements A

内部类：
分类：局部内部类，匿名内部类，成员内部类，静态内部类
局部内部类
通常定义在方法内部，可以访问外部类的所有成员，包括私有的
不能添加访问修饰符，因为局部内部类的地位是一个局部变量，但可以添加 final
作用域仅仅是定义它的方法或代码块
外部类在方法中可以创建内部类的对象
局部内部类的地位是一个局部变量，所以外部其他类不能访问内部类
如果外部类和内部类的成员重名，那么遵循就近原则，如果要调用外部类的成员，就用外部类名.this.成员名表示（外部类名.this 本质就是外部类 new 出来的一个对象）
匿名内部类（最重要）
匿名内部类的使用需求是一个类创建一个对象之后就不想用了，可以简化开发
匿名内部类的名字是系统分配的，运行类型是外部类$1 不仅是一个类，还是一个对象
匿名内部类使用一次就不能再使用
jdk 底层在创建了匿名内部类之后立刻创建了实例，并把地址返回
匿名内部类可以当做实参传递
成员内部类
作用域和外部类的其他成员一样，可以用各种权限修饰符来修饰
其他外部类也可以访问成员内部类
第一种方式：
cellPhone c=new cellPhone();
cellPhone.phone ph=c.new phone();
第二种：在外部类中编写一个方法返回成员内部类对象
静态内部类
可以直接访问外部类的所有静态成员
作用域和成员内部类一样，都是整个外部类
外部其他类要访问内部类，可以直接通过类名访问比如：
Outer.Inner inner=new Outer.Inner();
也可以编写一个返回静态内部类的方法
如果外部类成员和内部类发生冲突，要想访问外部类成员，直接外部类名.成员名就可以

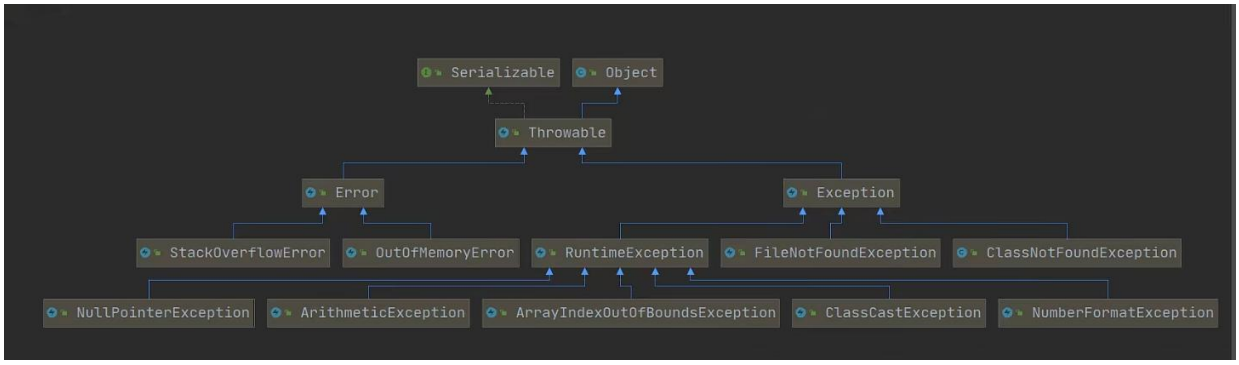
基于面向对象的应用

枚举和注解：
如果按照传统的写类的思路来写确定有固定几个的对象是不合适的
把对象一个个列出来的类成为枚举类，枚举类是特殊的类，包含有限的特定的对象
自定义枚举类的实现：1.构造器私有化 2.去掉 set 相关的方法，防止属性被修改 3.本类内部创建一组对象 4.对外暴露对象，添加 public/static/final 修饰符 5.对外提供 get 方法
enum 关键字实现枚举类：1.class 关键字改为 enum 2.常量名（实参列表）的格式写在最前面 3.多个常量之间逗号隔开
枚举类的所有实例都必须放在第一行展示，不需使用 new 关键字，不需显式调用构造器。
自动添加 public static final 修饰。
属性要设置成 final 的，否则会被修改

枚举对象的父类是 Enum，枚举类的 toString 返回的是类名
Enum 类的常见方法：
ordinal 返回的是枚举类的编号
类名.values 返回的是一个包含所有枚举类的数组
类名.valuesof(已有的常量名） 将字符串转换为枚举类对象
compareTo 比较两个类名的编号，返回的是两个编号的差

使用 enum 之后就不能继承其他类了，因为 enum 就是继承的 Enum 类
枚举类和普通类一样可以实现接口

异常 Exception



ClassCastException 类型转换异常，试图将对象强转为不是子类的类时，出现该异常
异常处理方式：
try-catch-finally:finally 里是一定发生的，一般将释放资源的代码放入 finally
throws:把异常抛给上一级，最顶级的是 JVM，JVM 的处理方式就是直接输出异常信息，退出程序
可以有多个 catch 语句捕获多个异常，要求父类异常在后，子类异常在先
在方法声明中用 throws 语句声明抛出类型的列表，throws 后面抛出的异常可以是方法产生的异常，也可以是这个异常的父类，也可以是异常列表
对于运行异常，默认处理是 throws；编译异常必须进行处理
子类重写方法时，跑出的异常要么是父类异常类型，要么是父类异常的子类型
自定义异常

常用类
包装类
手动装箱和手动拆箱：Integer n2=Integer.valueOf(n1);
int n3=n2.intValue();
jdk5 以后自动装箱和自动拆箱：Integer n=n1;
int n2=n;

```
//示例六
Integer i11=127;
int i12=127;
//只有有基本数据类型，判断的是
//值是否相同
System.out.println(i11==i12); //T
//示例七
Integer i13=128;
int i14=128;
System.out.println(i13==i14);//T
```

String 类
一旦一个类实现了 Serializable 接口，就是串行化的，可以在网络传输
实现了 Comperable 接口，就可以比较大小
String 重写了很多构造器
String 有 value 属性，是 static 的，不可修改指的是指向的地址不可变，但是改地址的值可以被改变
如果 new 一个 String 对象，引用会先指向堆，堆中的信息再指向常量池。intern 会返回这个 String 在常量池中的地址

String 和 char[]的转化：String 变成 char[]只需要.toCharArray
char[]变成 String 可以 new String(charshuzuming)

String arr="hello";
String arr2=" jrh";
String arr3=arr+arr2;
以上创建 arr3 执行具体流程：1.创建 StringBuilder sb 2.sb.append(“hello”) 3.sb.append(“ jrh”) 4.arr3=sb.toString();　　所以 arr3 指向的实际上是堆中的内容
所以常量相加结果指向常量池，变量相加结果指向堆

```
public class Test1 {
    String str = new String("hsp");
    final char[] ch = { 'j', 'a', 'v', 'a'};
    public void change(String str, char ch[]) {
        str = "java";
        ch[0] = 'h';
    }
    public static void main(String[] args) {
        Test1 ex = new Test1();
        ex.change(ex.str, ex.ch);
        System.out.print(ex.str + " and ");
        System.out.println(ex.ch);
    }
} // 思考，认真看，仔细想 3min
```

这个题，非常细节

StringBuffer：

直接继承 AbstractStringBuilder

String 和 StringBuffer 内容都存在于 char[] value 当中，但 String 的 value 是 final 的，存放在常量池，不可修改，而 StringBuffer 的 value 存放在堆中，可以修改

每次 String 更新内容都是在修改地址，而 StringBuffer 不需要每次修改

new 一个 StringBuffer，默认情况下 value 数组长度为 16，也可以将希望的长度转作为参数，如果放入一个字符串作为参数，那么 value 长度初始为字符串长度+16

String 转换成 StringBuffer 可以用 append 方法

StringBuffer 转换成 Strign 可以用 toString 方法

StringBuffer 常用方法：append,delete,replace,indefOf,insert,length

```
看下面的代码输出什么? 为什么? 2min

String str = null;// ok
StringBuffer sb = new StringBuffer(); //ok
sb.append(str);//需要看源码，底层调用的是 AbstractStringBuilder 的 appendNull
System.out.println(sb.length());//4

System.out.println(sb);//null
//下面的构造器，会抛出NullpointerException
StringBuffer sb1 = new StringBuffer(str);//看底层源码 super(str.length() + 16);
System.out.println(sb1);
```

StringBuilder 类

提供与 StringBuffer 兼容的 API，但不是线程安全的。单线程的情况下它的运行速度快，但多线程情况下 StringBuffer 更安全

Arrays 类

Arrays.toString()

Arrays.binarySearch()的返回值，如果找到则返回位置，没找到，返回的是如果存在于数组中应在的位置+1 再取反

Arrays.copyOf(arr,length)

Arrays.fill(arr,num)使用 num 替换 arr 中所有元素

Arrays.asList(一组数) 将一组数装到 list 当中，返回的内容编译类型是 List，运行类型是

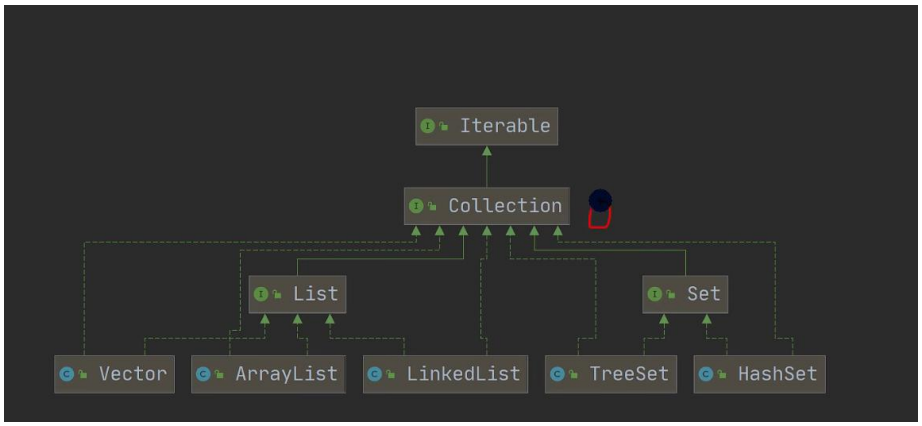
ArrayList

日期

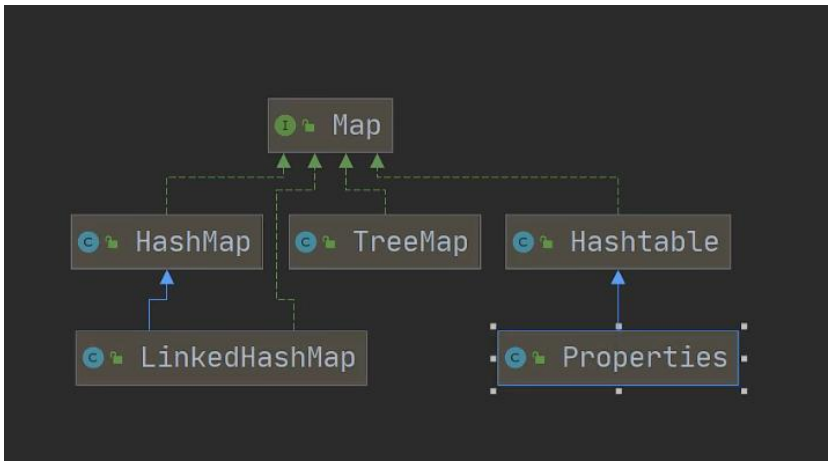
```
Date d1=new Date();
System.out.println(d1);
SimpleDateFormat sdf=new SimpleDateFormat("yyyy 年 MM 月 dd 日  hh:mm:ss E");
String format=sdf.format(d1);
System.out.println(format);

Calendar c=Calendar.getInstance();//Calendar is abstract,can not new
System.out.println(c.get(Calendar.YEAR)+" "+(c.get(Calendar.MONTH)+1));
//月份是从零开始编号，所以要+1
//Calendar 只能由自己组合
System.out.println(c.get(Calendar.HOUR_OF_DAY));//24hours 制
//LocalDateTime 类
LocalDateTime ldt=LocalDateTime.now();
System.out.println(ldt);
System.out.println(ldt.getYear());
System.out.println(ldt.getDayOfYear());
//DateTimeFormatter 类
DateTimeFormatter dtf=DateTimeFormatter.ofPattern("yyyy 年 MM 月 dd 日  HH 小时 mm 分 ss 秒");
String f=dtf.format(ldt);
System.out.println(f);
```

集合类

好处是可以保存对象的数量灵活，提供了多个操作的方法



单列集合



双列集合

List 接口的实现类：

Collection 的存放是有序的，Set 是无序的

remove 是按索引删除

addAll 和 removeAll 的对象都是一个集合，也就是从当前集合中添加或删除另一个集合所有元素

添加元素也可以以索引值做参数，表示在哪个位置添加元素

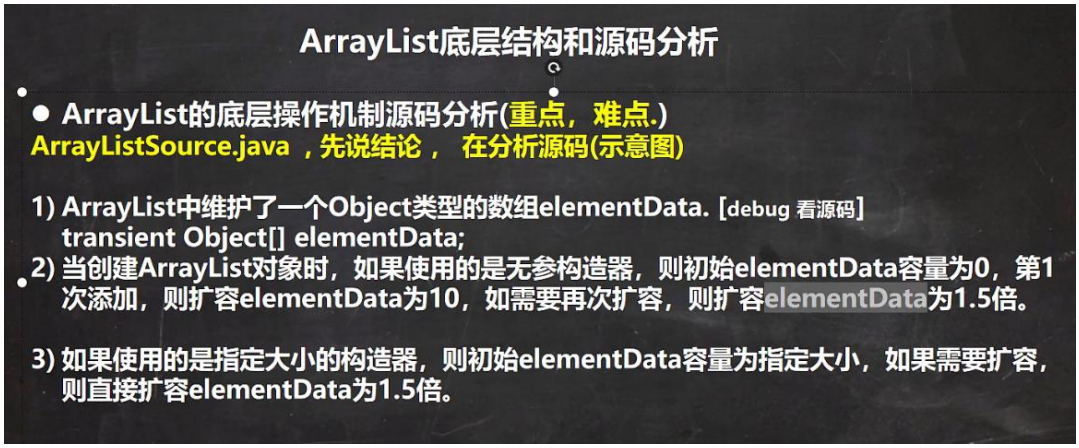
al.lastIndexOf(9)　　al.lastIndexOf(9)

al.set(0, 8);相当于 replace

al.subList(0, 2)

ArrayList 可以加入多个 null

ArrayList 是线程不安全的，执行效率比较高



ArrayList 源码分析：

1. 创建一个空的 elementdata 数组
2. 添加元素时，先确定是否先扩容，再确定 minCapacity，第一次扩容为 10
3. 在 ensureExplicitCapacity 中，modCount++，记录被修改的次数

Vector 是线程同步的，和 ArrayList 相比，是二倍扩容

LinkedList 底层是一个双向链表，而 ArrayList 底层则是一个 object 数组，二者都线程不安全
所以 LinkedList 在添加和删除元素的效率比较高，ArrayList 数组扩容需要 copy 所以效率低
但是改和查的效率上，ArrayList 效率更高
LinkedList 的 set 方法，对于索引值小于 size/2 的从 first 开始遍历，反之则从 last 开始遍历

Set 接口的实现类：
无序的，无索引的，不可有重复数据，最多一个空值
和 List 接口一样，都是 Collection 的子接口（所以都有 Iterator），获得元素的方式可以用迭代器，可以增强 for 循环（底层就是迭代器），但无法我那个索引
HashSet 的底层其实是 HashMap，链表+数组+红黑树
执行 add 方法，返回的是一个 boolean 值，表示是否添加成功

HashSet 的去重机制：先存入一个对象，得到 hash 值，通过 hash 值找到对应索引，如果索引没有值就直接添加，如果有值，就用 equals 进行循环比较，若返回都为 false 则将该对象加到链表结尾

HashMap 的原理：添加一个元素时，会先得到 hash 值，然后转换成索引值，找到 table，如果要添加的元素不存在则直接加入，如果存在则调用 equals，相等则不添加
如果一条链表中元素到 8，并且整个表的元素个数到 64，那么这条链会变成红黑树；如果表的元素没到 64，就对哈希表进行扩容
hash 值是怎样得到的：在 add 方法中存在一个 put 方法，put 方法里面有个 putVal，需要求 h=(key.hashCode())^(h>>>16);按位异或
putVal 方法：1.先定义辅助变量 2.table 是 HashMap 放结点的一个数组，如果这个数组是空的，就扩容到 16 3.根据 key，得到 hash，计算 key 应该存到 table 的哪个位置（利用 (n-1)&hash）,并且把这个位置的对象赋给 p；如果 p 为空，表示当前无数据，就创建一个 Node
如果 p 不为空：①如果准备加入的 key 和 p 指向的 node 的 key 相同，或者 equals，就让 e=p ②如果 p 是红黑树③否则用 for 循环依次比较，如果都不相同，则加入到该链表最后，随机

判断该链表是否达到 8 个结点，达到则进行红黑树化
4.判断添加之后，数量是否超过临界值（容量*0.75）,达到就扩容
如何让自定义的类根据属性相同返回相同的 hashcode（重写 equals 和 hashCode 函数）：

```
public boolean equals(Object o) {
    Employee e=(Employee)o;
    if(name.equals(e.name) && age==e.age)
        return true;
    return false;
}

public int hashCode() {
    return Objects.hash(name,age);
}
```

LinkedHashSet 是 HashSet 的子类，底层是 LinkedHashMap，维护的是一个数组和双向链表
插入的顺序和遍历的顺序一致
添加第一次时，直接将数组 table 扩展到 16，数组中的节点类型是 LinkedHashMap$Entry

TreeSet：
可以是有序的，但如果使用无参构造器，仍然是无序的
使用 TreeSet 提供的构造器，可以传入一个比较器：

```
TreeSet treeSet=new TreeSet(new Comparator() {
        public int compare(Object o1,Object o2) {
            return ((String)o2).compareTo((String)o1);
        }
    });
```

TreeSet 添加元素时，如果没有传入比较器，一定会把对象转成 comparable 类型，如果这个对象没有 compareTo 方法，一定会报错
要解决这个问题可以让这个对象对应的类实现 Comparable 接口，并重写 CompareTo 接口

如果重写了一个类的 equals 和 hashCode 方法，那么这个改变对象的属性，会导致 hashCode 改变，因此这个元素在 set 中的位置也很可能改变

TreeSet 和 HashSet 的区别：
https://blog.csdn.net/lucky_love816/article/details/115534265

Map 接口的实现
存储的是键值对，实际上 Set 也是存储的键值对，但 set 中的 value 都是常量替代的
键值对就是用来保存具有映射关系的对象
Map 中的 key 和 value 可以是任何数据类型，被封装到 HashMap$Node 对象中
元素也不能重复，原因同 HashSet，而 value 可以重复
当 put 一个新的键值对的时候，如果 key 和之前的元素相同，那么相当于替换原来的元素
map 中的 key 和 value 可以为空，但 key 为空只能有一个
key 和 value 单向一对一，用 get（key）方法可以获得对应的 value

```
//1. k-v 最后是 HashMap$Node node = newNode(hash, key, value, null)
//2. k-v 为了方便程序员的遍历，还会 创建 EntrySet 集合 ，该集合存放的元素的类型 Entry, 而一个Entry
//   对象就有k,v EntrySet<Entry<K,V>> 即： transient Set<Map.Entry<K,V>> entrySet;
//3. entrySet 中， 定义的类型是 Map.Entry ，但是实际上存放的还是 HashMap$Node
//   这时因为 static class Node<K,V> implements Map.Entry<K,V>
```

创建 Map 的时候会创建 table 和 EntrySet，EntrySet 里的结点还会指向 table 中的内容

因为 HashMap$Node 类型 implements 了 Entry，所以可以把 set 中的元素转成 Entry，如果把一个 EntrySet 中的元素 getClass

Map 接口常用方法：put，remove，get，size，isEmpty，clear，containsKey

Map 不是 implements 了 Collection 接口的接口，所以不能使用迭代器，但是 Map 里面的 EntrySet、Values、KeySet 这三个类都是实现了 Collection 的类，所以可以他们含有迭代器，因此遍历 map 中元素的方法有六种

Map 接口的常用实现类：HashMap，HashTable，Properties

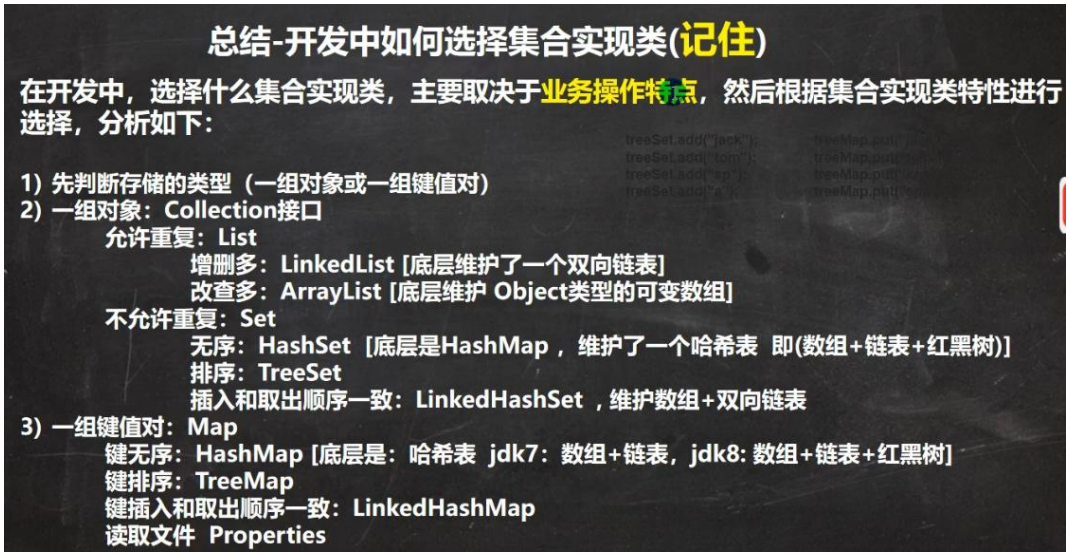
HashMap 线程不安全

HashTable 键和值都不可以为空，使用方法基本和 HashMap 一样，它是线程安全的

底层有一个数组 table，初始化大小为 11，每次扩容是*2+1

HashTable 的子类 Properties：

可以读取 xxx.properties 文件



Collections 常用工具类

1) reverse(List)：反转 List 中元素的顺序
2) shuffle(List)：对 List 集合元素进行随机排序
3) sort(List)：根据元素的自然顺序对指定 List 集合元素按升序排序
4) sort(List，Comparator)：根据指定的 Comparator 产生的顺序对 List 集合元素进行排序
5) swap(List，int， int)：将指定 list 集合中的 i 处元素和 j 处元素进行交换

```
//Object max(Collection)：根据元素的自然顺序，返回给定集合中的最大元素

//Object max(Collection，Comparator)：根据 Comparator 指定的顺序，返回给定集合中的最大元素
//Object min(Collection)
//        Object min(Collection，Comparator)
//int frequency(Collection，Object)：返回指定集合中指定元素的出现次数
//void copy(List dest,List src)：将src中的内容复制到dest中
//boolean replaceAll(List list，Object oldVal，Object newVal)：使用新值替换 List 对象的所有旧值
```

一般情况下，max、sort 都是按照自然顺序，如果想按自己的方法，就要通过匿名内部类重写比较方法，以 max 为例：

```
Collections.max(list,new Comparator() {
    @Override
    public int Compare(Object o1,Object o2) {
        if(o1 instanceof String && o2 instanceof String)
            return ((String)o1).length()-((String)o2).length();
        throw new ClassCastException();
        return;
    }
});
```

copy 方法必须保证 dest 大小不小于 src 否则就会抛出异常

泛型
表示数据类型的数据类型，又称为参数类型
可以在类声明时，通过一个标识符表示类中某个属性的类型，或者是某个方法的返回值的类型，或者是参数类型，这个标志符代表的数据类型是在定义对象时指定的
泛型可以有多个，接口也可以使用泛型
泛型只能放引用数据类型，不能放基本数据类型
指定泛型后，可以放入子类类型
可简写，推荐使用：List<Integer> list=new ArrayList<>();
如果没写泛型是什么类型，那么默认为 Object
使用泛型的数组不能初始化，不能被 new
使用泛型的方法也不能是 static 的，因为类的加载早于对象创建
要区分开泛型方法和使用泛型的方法
泛型不具备继承关系：
List<List> arrayList=new ArrayList<ArrayList>();这样写就不对
泛型的通配符：<?>支持任意类型的泛型
<?extends A> A 类以及 A 的子类
<?super A>A 类以及其父类
List<?extends List> arrayList=new ArrayList<ArrayList>();这样写就对了

java 绘图坐标体系
x 轴向右，y 轴向下
1.定义一个 panel 继承 JPanel 类　　2.重写 paint 方法
panel 是一个面板，paint 的参数类型 Graphics 是画笔，提供了许多绘图的方法
3.用 Graphics 的对象 g 的 drawOval 方法画一个椭圆
4.含有 main 函数的 class 继承 JFrame，在这个 class 中定义一个面板
paint 方法什么时候被调用：组件第一次在屏幕显示时、窗口被最小化后放大时、窗口大小发生变化时、repaint 方法被调用时

paint 方法里面常用的方法

```
super.paint(g);//调用父类方法完成初始化
        g.drawLine(100, 100, 180, 180);
        g.drawLine(180, 100, 100, 180);
        g.setColor(Color.DARK_GRAY);
        g.fillRect(100, 100,80, 80);
        g.setColor(Color.MAGENTA);
        g.fillOval(100, 100, 80, 80);
        g.setColor(Color.red);
        g.setFont(new Font("宋体",Font.BOLD,25));
        g.drawString("你好",115, 200);

        Image image=Toolkit.getDefaultToolkit().getImage(
        Panel.class.getResource("/rbx.png"));
        g.drawImage(image,10,10,130,64,this);
```

设置面板：

```
        mp=new MyPanel();
        this.add(mp);//把面板放入画框
        this.setSize(600,500);//画框大小 400*300
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);//点击窗口叉号时程序退出
        this.setVisible(true);
```

在继承 JFrame 的类中使用 addKeyListener，别的没有绘图的地方还是用 scanner 吧

多线程
一个类继承了 Thread 类，就可以当成线程使用
run 方法是 Thread 类实现接口 runnable 来的
主线程结束了，并不意味着其他线程也要结束
为什么调用 start 来开始线程，而不是 run：如果是 run 那就是串行化执行主线程和子线程
start 底层执行是调用 start0 方法，start0 是一个 native 方法（本地方法），是由 JVM 执行的
使用继承 Thread 和实现 Runnable 没有本质区别，但是后者可多个线程共享统一资源

线程常用方法

- 常用方法第一组

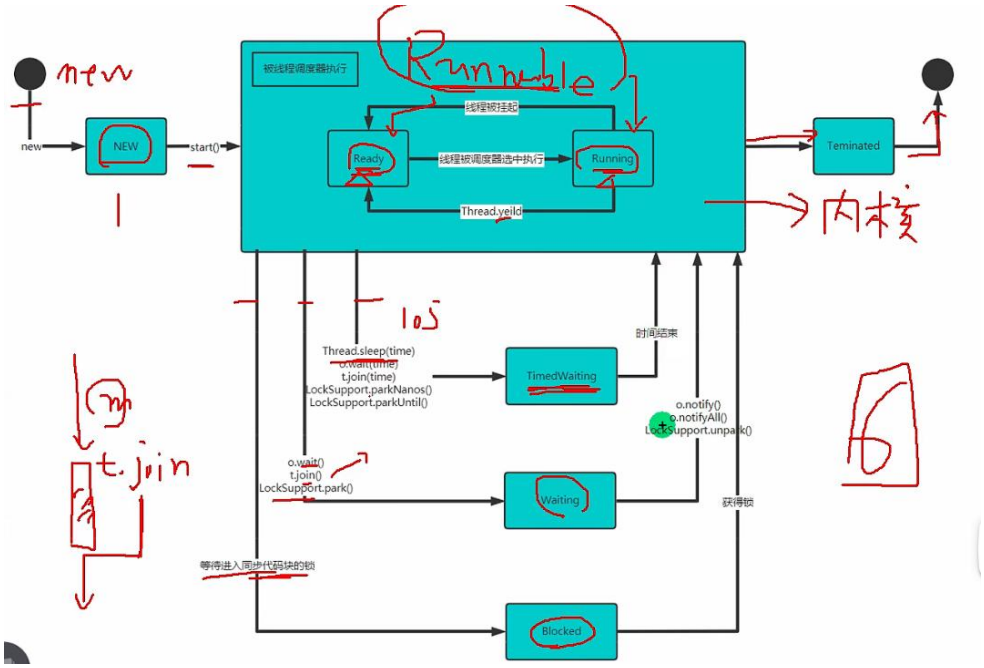
1. setName //设置线程名称，使之与参数 name 相同
2. getName //返回该线程的名称
3. start //使该线程开始执行；Java 虚拟机底层调用该线程的 start0 方法
4. run //调用线程对象 run 方法；
5. setPriority //更改线程的优先级
6. getPriority //获取线程的优先级
7. sleep //在指定的毫秒数内让当前正在执行的线程休眠（暂停执行）
8. interrupt //中断线程

- 常用方法第二组

1. yield:线程的礼让。让出cpu，让其他线程执行，但礼让的时间不确定，所以也不一定礼让成功

2. join：线程的插队。插队的线程一旦插队成功，则肯定先执行完插入的线程所有的任务
案例：创建一个子线程，每隔1s 输出 hello，输出 20 次，主线程每隔1秒，输出 hi，输出 20次.要求：两个线程同时执行，当主线程输出 5次后，就让子线程运行完毕，主线程再继续，

用户线程和守护线程：
用户线程也叫工作线程，守护线程为工作线程服务，工作线程全结束时，守护线程也结束
将一个线程设置成守护线程：setDaemon 方法（在启动之前设置好）
线程的七种状态



synchronize 关键字，可以直接修饰方法，也可以修饰代码块
非静态方法 synchronize 修饰的是 this，静态方法 synchronize 修饰的是类，使用方法如下：

```
public static void m() {
    synchronized(Tickets.class) {
    }
}
```

线程的死锁：多个线程占用了对方的锁资源
会释放锁的情况：同步方法/同步代码块执行结束，执行过程遇到 break、return，出现了未处理的 error 或 exception，执行了 wait 方法
不会释放锁的情况：sleep、yield 方法，其他线程调用了 suspend 方法
但是目前 suspend 和 resume 都不建议使用了

IO 流：
磁盘到内存是输入流，内存到磁盘是输出流
new File 之后只是得到一个对象而已，只有 file.createNewFile 之后才能真正在磁盘创建文件
File 常见方法：getName、getAbsolutePath、getParent、length（单位是字节数）、exist、isFile、isDirectory
在 java 中，目录被看成特殊的文件，创建一级目录：file.mkdir,多级目录：file.mkdirs
流的分类：字节流和字符流，字节流在操作二进制文件是可以无损操作，字符流适合操作文本文件

(抽象基类)	字节流	字符流
输入流	InputStream	Reader
输出流	OutputStream	Writer

以上四个都是抽象类
字节流
FileInputStream，常见方法 read（）：读取单个字符，效率比较低
read(byte[] b):从输入流中读取最多 b.length 长度的字节数组，文件到达结尾返回-1，如果读取正常，返回实际读取的 byte 长度
FileOutputStream，如果使用 new FileOutputStream(FileName)的方式打开文件，原来的内容会被覆盖,而 new FileOutputStream(FileName,true)的方式，就是追加，不会覆盖
文件拷贝：
不要把整个文件放到内存之后再移动，否则内存会不够
文件拷贝的 dest 就算不存在，在拷贝的过程中也会创建它，因为在创建 FileOutputStream 对象时就会创建文件

字符流的 FileReader 和 FileWriter
read 方法读取单个字符，而不是单个字节，所以汉字就不会出现乱码
read(char[]),不是 byte 数组了
FileWriter 在执行完 write 之后一定要关闭流或者刷新（flush），否则无法写入

分　类	字节输入流	字节输出流	字符输入流	字符输出流
抽象基类	*InputStream*	*OutputStream*	*Reader*	*Writer*
访问文件	**FileInputStream**	**FileOutputStream**	**FileReader**	**FileWriter**
访问数组	**ByteArrayInputStream**	**ByteArrayOutputStream**	**CharArrayReader**	**CharArrayWriter**
访问管道	**PipedInputStream**	**PipedOutputStream**	**PipedReader**	**PipedWriter**
访问字符串			**StringReader**	**StringWriter**
缓冲流	BufferedInputStream	BufferedOutputStream	BufferedReader	BufferedWriter
转换流			InputStreamReader	OutputStreamWriter
对象流	ObjectInputStream	ObjectOutputStream		
抽象基类	*FilterInputStream*	*FilterOutputStream*	*FilterReader*	*FilterWriter*
打印流		PrintStream		PrintWriter
推回输入流	PushbackInputStream		PushbackReader	
特殊流	DataInputStream	DataOutputStream		

节点流

处理流

处理流
节点流是底层流，处理流包装节点流，可以实现不同节点流的实现差异
只要是 Reader 的子类，都可以包装到 BufferedReader 里面
bufferedReader 的 read 方法可以逐个字符读取（返回的是索引），也可以读一行（返回的是字符串）

对象流：
可以将数据类型序列化和反序列化，也就是可以保存和恢复数据类型和它的值
要想让某个类是序列化的，必须实现 Serializable 或 Externalizable 接口之一
Serializable 是一个标记接口，里面没有任何方法
ObjectInputStream 实现反序列化 ObjectOutputStream 实现序列化
对对象进行反序列化的时候，输入流必须能看到反序列化的类
读写顺序要一致
对象序列化时，默认对象都序列化，但 transient 和 static 的不序列化
序列化对象时，要求里面被序列化的对象所属的类也是序列化的

标准输入输出流
System.in 的变异类型是 InputStream，运行类型是 BufferedInputStream
System.out 编译类型是 PrintStream，运行类型是 PrintStream

字节流和字符流的转化
默认的编码类型是 utf-8，如果编码方式不符合就会乱码，所以要指定类型
InputStreamReader，OutputStreamWriter 可以将字节流包装成字符流，也可以指定编码方式

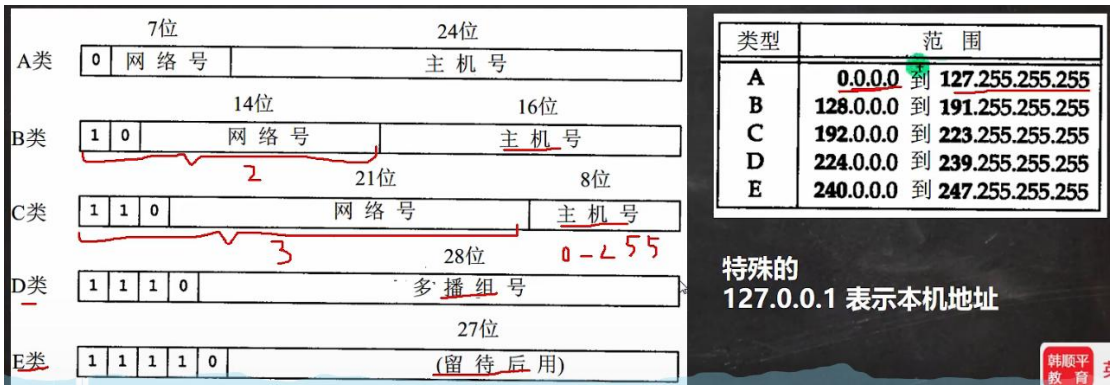
打印流
打印流只有输出流，没有输入流
打印流默认的输出就是标准输出，也就是控制台
print 方法底层实现是 write
//可以修改打印流输出的位置
System.setOut(new PrintStream("d:/java/untitled/date.txt"));
之后的输出都会输出到目标文件中

Properties
将配置文件的内容读入到程序当中，或者从程序将修改内容写回到配置文件
Properties 类是 HashTable 下的子类

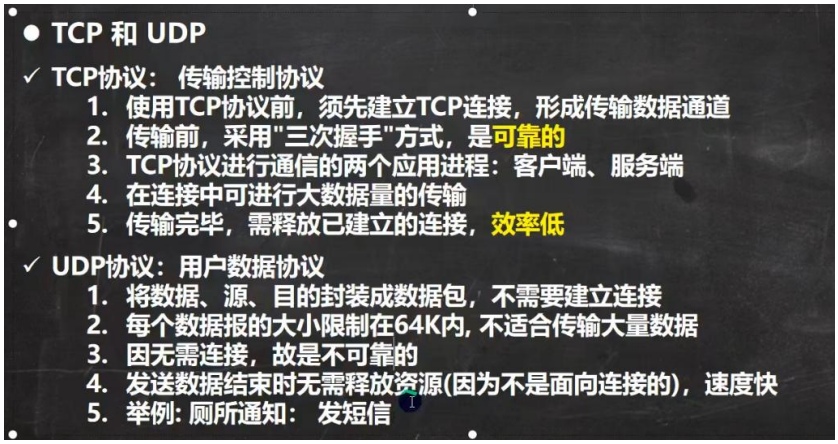
常用方法：load　将配置文件的键值对加载到 Properties 对象
list　将键值对显示到指定的设备、流对象
getProperty（key）　　根据键获取值
setProperties（key）　　修改
store　　存储

网络编程

基础知识
java.net 提供一系列类和接口，用于完成网络通信
ip 地址：用于唯一的标识一台主机
IPv4 地址用四个字节表示（点分十进制），最大的问题在于资源有限，IPv6 是替代它的下一代 IP 协议，IPv6 使用 128 位（16 字节）表示地址（冒分十六进制）
IP 地址=网络地址+主机地址
IPv4 的类型和范围



域名：IP 地址的映射，更好记
端口用来表示主机中的特定的网络程序，访问其他主机的程序，需要 IP+端口，端口占用 2 字节，范围是 0——65535，0 到 1024 已经被占用，比如 ssh-22，http-80，ftp-21
网络编程中，数据的组织形式，就是协议
传输层的 TCP 协议和 UDP 协议：
使用 TCP 协议之前，先建立 TCP 连接，形成数据传输通道；传输前采用三次握手
TCP 由于三次握手，可靠性更高，比较适合大量数据，传输完需要释放建立的连接



InetAddress 类
常用方法：
InetAddress address=InetAddress.getLocalHost();//计算机名+IP 地址
InetAddress host=InetAddress.getByName("laptop-rommel");//根据计算机名，获取 InetAddress
InetAddress baidu=InetAddress.getByName("www.bilibili.com");//根据域名得 InetAddress 对象
String hostAddress=baidu.getHostAddress();//根据对象获取地址
String hostName=baidu.getHostName();

Socket 是数据通道的两端，一头是客户端，一头是服务器端，客户端发起连接，服务器端接受请求连接
客户端的行为：1.连接服务器（IP 和端口）2.连接上之后生成 socket，通过 socket.getOutputStream() 3.通过输出流，写数据到数据通道
服务器端行为：1.在本机端口监听，等待连接 2.当没有客户端连接端口时，会发生阻塞，等待连接 3.通过 socket.getInputStream()读取客户端写入的数据
客户端行为：

```
//本机 IP+域名，意思是连接本机的 9999 端口，如果成功，等待连接
Socket socket=new Socket(InetAddress.getLocalHost(),9199);
System.out.println("connecting...");
OutputStream stream=socket.getOutputStream();
stream.write("hello,server".getBytes());
stream.close();
socket.close();
System.out.println("clieant exit");
```

服务器端行为：

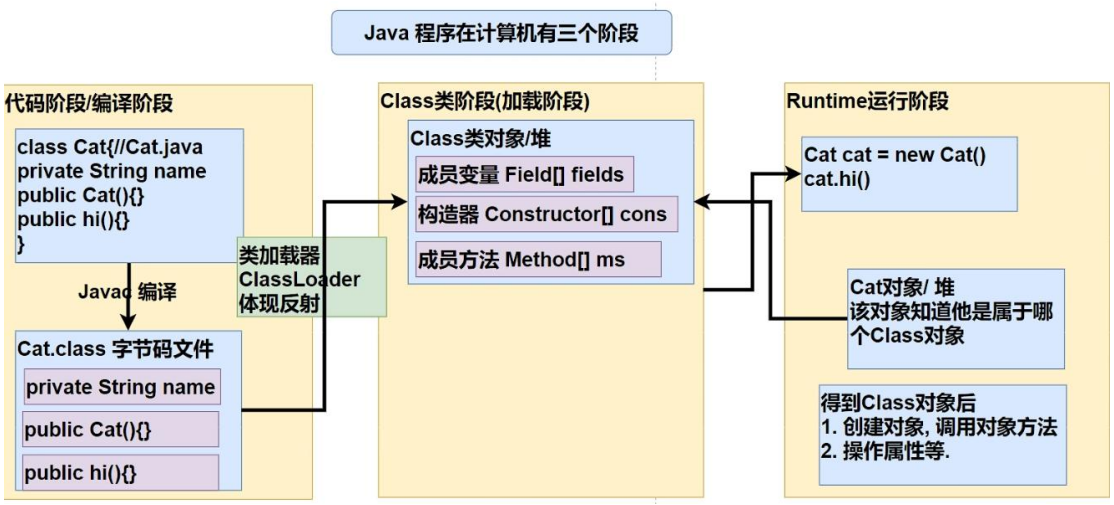
```
ServerSocket server=new ServerSocket(9199);//在 9199 端口（要求没有其他服务在监听 9999，否则失败）
//此刻 9999 端口已被占有
System.out.println("waiting...");
Socket socket=server.accept();//如果有客户端连接，程序继续执行\
//ServerSocket 可以通过 accept 返回多个 socket
System.out.println("server get"+server.getClass());

InputStream stream=socket.getInputStream();
byte[] buf=new byte[1024];
int readLen=0;
while((readLen=stream.read(buf))!=-1) {
    System.out.println(new String(buf,0,readLen));
}
stream.close();
socket.close();
server.close();
```

反射

反射是框架的底层机制，是框架的灵魂
有需求引入反射：需要通过外部文件配置，在不修改源码的情况下控制程序
示例：从 mysql.Properties 文件中获取类和方法，并实例化类，调用类中的方法

```
String fileName="d:/java/untitled/mysql.properties";

Properties properties = new Properties();

properties.load(new FileInputStream(fileName));

String className=properties.getProperty("class");

String methodName=properties.getProperty("method");

Class<?> aClass = Class.forName(className);

Object o = aClass.newInstance();//得到类的对象实例

Method method = aClass.getMethod(methodName);//可以把方法视为对象

method.invoke(o);//方法.invoke（对象）
```

一个对象经过编译，得到字节码文件，里面含有所有成员
之后这个对象经过类加载器 ClassLoader，加载到堆中，这里就体现了反射



想要获得类中的属性：
属性权限必须是当前方法能够访问到的，否则找不到

```
String name1 = properties.getProperty("name");
```

```
Field name = aClass.getField(name1);
```

```
System.out.println(name.get(o));
```

反射是解释性执行，运行速度比传统方法慢

class 类
不管是传统的调用类的方法，还是使用反射，都会调用到 classLoader 中的 loadClass 方法
对于某个类的 class 类对象，内存中只有一份，系统中只加载一次

```
Class<?> cls = Class.forName(className);
```

```
System.out.println(cls);//输出的是 com.socket.java.Dog
```

```
System.out.println(cls.getClass());//输出 java.lang.Class
```

获取 class 对象的方式：（利用的是程序加载阶段不同）
1. 在编译阶段可以用 class.forName
2. 加载阶段可以用类.class，这种途径多用于参数传递
3. 运行阶段可以用对象.getClass
4. 类加载器加载的过程中获得

哪些东西有 class 对象：外部类，内部类，接口，枚举，注解，数组，基本数据类型，void
（插播一下：Thread.state 是枚举类）
静态加载和动态加载：
静态加载依赖性强，在编译时进行，如果运行了不存在的类会报错
动态加载只有真正运行的时候才进行
类加载的三个阶段：加载、连接（包括验证、准备和解析）、初始化
加载：将类的 class 文件读入内存，创建 java.lang.Class 对象，有加载器完成
连接：将二进制数据合并到 JRE
初始化：对类进行初始化
创建对象的几种方式：

```
Class<?> cls = Class.forName("com.reflection_.java.User");
//无参构造器获取
User user=(User) cls.newInstance();
//有参构造器获取
Constructor<?> constructor = cls.getConstructor(String.class);
Object o = constructor.newInstance("hell");
//通过非 public 的有参构造器
Constructor<?> declaredConstructor = cls.getDeclaredConstructor(int.class, String.class);
declaredConstructor.setAccessible(true);//暴破，使用反射，可以使用私有的成员
Object hill = declaredConstructor.newInstance(40, "hill");
```

调用方法：

```
Method toString = cls.getDeclaredMethod("toString");
toString.invoke(user);
```